

Strategy_Vehicle_ver1.Vehicle 詳細解説

対象実装: MODEL3/Strategy_Vehicle_ver1.py (`class Vehicle(Vehicle_BASE)`)

この文書は、`Strategy_Vehicle_ver1.Vehicle` が実装している以下を、**コードの挙動に即して**整理した解説です。

- どのタスクを「放出候補 (offer)」として選び、どの車両へ提案するか (`offer_on_negotiation`)
- 交渉で得た合意 (`Agree`) を、どれだけ実行候補として残すか (`sign_contracts`)
- 合意のよし悪しを評価するコスト関数 (`calculate_cost_saving`) の内訳・符号・進行度依存
- ルート挿入 (`least_cost_time_insertion_index`) と、時間窓 (Time Window) を使った近似的な実現可能性チェック
- RL ルートプランナー統合 (`route_planner`) と、Q 重要度 (タスク重要度) による補正

関連 (前提として参照されるもの) :

- MODEL3/Vehicle.py (`Vehicle_BASE` の状態・掲示板・RLフック)
- MODEL3/classes.py (`Task`, `Offer`, `Agree`, `Bulletin`, `pac_task`)
- MODEL3/VRPTW_functions.py (`earliest_start_time_list`, `latest_start_time_list`, `euclidean_distance` など)
- MODEL3/bulletin_are_search.py (`calculate_dynamic_area`)
- MODEL3/Vehicle_Negotiator.py (交渉中の提案列・受諾判定。署名と評価式のズレに注意)

0. データ構造と前提

0.1 Task (顧客)

MODEL3/classes.py の `Task` は少なくとも以下の属性を持ちます。

- `id`: 顧客ID
- `x_coordinate`, `y_coordinate`: 座標
- `weight`: 需要 (積載)
- `ready_time`: サービス開始可能な最早時刻
- `due_date`: サービス開始期限 (この実装では「開始期限」として扱われる箇所が多い)
- `service_time`: サービス時間

0.2 pac_task (時刻計算付きタスクラップ)

MODEL3/classes.py の `pac_task` は `task: Task` を包み、追加で

- `earliest_start_time`
- `late_start_time`
- `earliest_arrival_time`

を保持します。`Strategy_Vehicle_ver1` は **実タスク列** `self.tasks: list[Task]` と並行して、**`self.arrival_time_list: list[pac_task]`** を持ち、スラックや遅れ (`over_window`) の計算に使います。

この `arrival_time_list` は、各ステップ冒頭の `first_step()` で作り直されます。

0.3 掲示板 `Bulletin`（協調のための共有状態）

`MODEL3/classes.py` の `Bulletin` は少なくとも次を持ちます。

- `time_board` (DataFrame) : 車両ごとの `slack_time`, `departure_time`, `return_time` など
- `area_board` (DataFrame) : 車両IDと、時間帯ごとの「最も滞在したエリア」など
- `n_steps`, `max_steps`: 交渉の進行度 (0~max)
- `X`, `n`: 空間分割のパラメータ (`calculate_dynamic_area` に渡される)
- `zones`: 時間帯（ゾーン）辞書 (`find_time_zone` に渡される)

0.4 車両状態 (`Vehicle_BASE` 由来)

`Strategy_Vehicle_ver1.Vehicle` は `Vehicle_BASE` を継承し、代表的には以下を利用します。

- `self.tasks`: 現在の担当タスク列（ルート）
- `self.max_weight`, `self.current_weight`
- `self.bulletin_board`
- `self.offer_nego_list`: そのステップに生成した `Offer` の蓄積先
- `self.over_task`: 交渉中に相手へ差し出す候補の優先順位 (`before_negotiation` が更新)

また、RL 統合用のフックとして

- `self.route_planner` (`set_route_planner / evaluate_route_with_planner`)
- `self.q_importance_weight` (Q重要度の重み)

を持ちます。

0.5 主な係数（実装に埋め込まれている値）

`Strategy_Vehicle_ver1` は複数の係数を「定数」として埋め込んでいます。チューニングや結果解釈の際はここが効きます。

- K-means:
 - `n_init=10`
 - `max_clusters = int(num_samples / 2)`
- 署名 (`sign_contracts`) の閾値:
 - `cost_border = 100000 +  $\alpha$ (progress)` (α はごく小さい)
- コスト関数 (`calculate_cost_saving`) :
 - `slack_late = 0.5`
 - `distance_late = 0.5`
 - `over_late = 10 * (n_steps/max_steps)^2`
- RL補正（クラス定数）:
 - `RL_REWARD_WEIGHT = 1.0`
 - `RL_UNASSIGNED_WEIGHT = 100000.0`
- 実ルート挿入 `add(...)` の容量緩和:
 - `max_weight + 10 * (max_steps - n_steps) / max_steps`

1. 交渉提案生成 `offer_on_negotiation(run_cars, offer_id, vehicles)`

1.1 目的と返回值

この関数は「このステップで、誰に何のタスクを提案して交渉を開始したいか」を列挙します。

- 返回值: `list[Offer]`
 - `Offer(id, vehicleA, vehicleB, task)` (`MODEL3/classes.py`)
 - 意味: `vehicleA` (自分) が `task` を手放したいので、`vehicleB` (相手) に交渉を持ちかける

実装上は `self.offer_nego_list` に追加し、最後にそれを返します。

1.2 近傍探索の材料 (空間×時間)

各タスクごとに、提案相手候補を次の2つで集めます。

1. 空間近傍 (area) :

- `task_area = calculate_dynamic_area(task.x_coordinate, task.y_coordinate, X, n)`
- `X` と `n` は掲示板 (`bulletin_board.X, bulletin_board.n`) から取得
- `calculate_dynamic_area` は空間を `n×n` のセルに分け、"A1" のようなラベルに変換します

2. 時間帯 (zone) :

- `task_time_zone = find_time_zone(task.ready_time, bulletin_board.zones)`
- `ready_time` が属するゾーン (A/B/C...) を返します

これらを使い、`find_vehicles_in_neighboring_areas(task_time_zone, task_area, bulletin_board.area_board)` で「時間帯×隣接エリア」内の車両IDを集めます。

補助として、`find_available_vehicles(bulletin_board.time_board, ready_time, due_date, vehicles)` も併用します (後述)。

1.3 タスクが少ない場合 (`len(self.tasks) < 3`)

少タスクのときは単純に「自分の全タスク」を放出候補にします。

疑似コード (実装の流れ) :

```
for task in self.tasks:
    candidates = neighboring_vehicles_by_area_and_time(task)
    candidates += find_available_vehicles_by_time_board(task)
    candidates -= {self.id}
    for each candidate:
        append Offer(self.id -> candidate.id, task)
return offer_nego_list
```

ポイント:

- 近傍探索 `find_vehicles_in_neighboring_areas` は **車両IDのリスト**を返す設計
- 一方 `find_available_vehicles` は **車両オブジェクトのリスト**を返す (後述)

- 少タスク分岐では `find_vehicle_by_id(vehicle, run_cars)` に「ID」を渡す前提のため、`find_available_vehicles` の返り値（オブジェクト）はそのままでは機能しづらい、という型のズレがあります

1.4 タスクが多い場合 (`len(self.tasks) >= 3`)

タスクが多いときは **K-means による「過密クラスタ検出」** を使って放出候補を絞ります。

(1) 座標の正規化

`MinMaxScaler` により `x, y` を 0~1 に正規化してクラスタリングします。

(2) クラスタ数 `k` の決め方（簡易エルボー）

SSE (`kmeans.inertia_`) を `k=1..max_clusters` で計算し、

$$\text{SSE変化率}(k) = (\text{SSE}(k-1) - \text{SSE}(k)) / \text{SSE}(k-1)$$

が最大となる `k` を採用します（実装上 +2 の補正を入れて `optimal_clusters` を算出）。

- `max_clusters = int(num_samples / 2)`
 - タスク数が3だと `max_clusters=1` → `k=1` のみ → `optimal_clusters=1`
 - その場合、後述の「過密クラスタ以外」が空になり、放出候補が0件になり得ます

(3) 「過密クラスタ以外」を放出候補にする

`Counter(task_clusters)` で各クラスタのタスク数を数え、最頻クラスタ (`most_common_cluster`) 以外に分類されたタスクを `other_cluster_tasks` に入れます。

直感的には、

- 最頻クラスタ**: その車両が「局所的にタスクを抱え込みすぎている（過密）エリア」
- それ以外のタスク**: ルート上の“外れ値”で、移動効率を悪くしている可能性がある

という見立てで、外れ値を手放そうとします。

(4) 積載が限界に近いときは「重いタスク」を追加

`self.current_weight >= self.max_weight` のとき、

- 平均重量 `wei = max_weight / len(tasks)`
- `task.weight >= wei` のタスクを `other_cluster_tasks` に追加（未追加なら）

として放出候補を増やします（容量圧迫の解消を意図）。

(5) 相手候補の収集と `Offer` の生成

多タスク分岐では、近傍探索で得た 車両IDを車両オブジェクトへ変換しています。

```
ids = find_vehicles_in_neighboring_areas(...)
objs = [vehiclesの中からidが一致するもの]
objs += find_available_vehicles(...)
objs = unique(objs) - {self}
for obj in objs:
    append Offer(self.id -> obj.id, task)
```

1.5 find_available_vehicles (時間板からの候補抽出)

find_available_vehicles(b_board, task_ready_time, task_due_date, vehicles) の条件は次です。

```
if row['departure_time'] > task_due_date or row['return_time'] <
task_ready_time:
    available_vehicles.append(row['id'])
```

解釈:

- 車両の稼働区間 (departure_time .. return_time) が、タスクの時間窓 (ready_time .. due_date) と **完全に前後でズレている車両**を拾います
- 「この時間帯に忙しくない (あるいは現在のルートがこの時間帯と無関係)」という粗いフィルタとして使われている可能性があります

注意:

- 返り値は「ID」ではなく「車両オブジェクト」で返しています (return [vehicle for vehicle in vehicles if ...])
- 少タスク分岐では候補の扱いがID前提なので、分岐間で整合していません

2. 署名 (実行候補の選別) sign_contracts(list: list[Agree])

このメソッドは、交渉の結果として得た Agree の集合から「この車両として実行してよい (実行したい) 合意」を返します。

2.1 合意のグルーピング (自分が手放すタスク単位)

各 Agree(vehicleA, vehicleB, taskA, taskB) について、

- taskA が self.tasks にあれば task = taskA
- そうでなければ taskB が self.tasks にあれば task = taskB

とし、「自車が手放す側のタスク」をキーに評価を蓄積します。

2.2 コスト計算と「タスクごとの最良候補」管理

合意ごとに cost = self.calculate_cost_saving(agreements) を計算し、

- min_cost[task] にコスト値を蓄積

- `min_cost_agreement[task]` に対応する `Agree` を蓄積

します。先頭 (`[0]`) が最小になるように「挿入 (`insert(0, ...)`) + 追加 (`append`)」で管理しています。

2.3 進行度による閾値 `cost_border`

各タスクについて「閾値を満たす合意」を `signed` に入れます。

```
if progress < 0.5:
    cost_border = 10 * (max_steps - n_steps) / max_steps + 100000
else:
    cost_border = -1 * (max_steps - n_steps) / max_steps + 100000
```

- どちらの分岐も **ベースが 100000** で、進行度でわずかに上下します
- `min_cost[task][i] < cost_border` を満たす合意は（重複していなければ）すべて署名候補に入ります

2.4 返り値の順序

一度 `signed`（内部では `cont(agree, cost)`）に集めた後、

- `cost` 昇順でソート
- `Agree` だけ取り出して返す

ので、返り値は「良さそうな (`cost` が小さい) 順」になっています。

2.5 実装上の注意（容量チェック）

署名候補の作成前に、次の条件でタスク単位にフィルタしています。

```
if self.current_weight + task.weight < self.max_weight + 0 * ...
```

これは実質 `self.current_weight + task.weight < self.max_weight` です。

ただしここでの `task` は「自分が持っている側のタスク」なので、交換によって **手放す** 場合でも `current_weight + task.weight` は増えてしまい、自然な制約式になっていません。（一般に `swap` の容量制約は `current - remove.weight + give.weight <= max` の形になります）

3. コスト関数 `calculate_cost_saving(agree: Agree)`

`Strategy_Vehicle_ver1` の中心はこの関数です。ここで返る値が「小さいほど良い（受け入れたい）」指標として使われます。

3.1 `remove_task/give_task` の解釈

- `remove_task`: 自分の `self.tasks` に **入っている方**（交換後に手放す側）
- `give_task`: 自分の `self.tasks` に **入っていない方**（交換後に受け取る側）

という形で決めています。

3.2 コストの大枠（加重和 + RL/Q補正）

実装を式にすると概ね次です（ $t = n_steps / max_steps$ ）。

```
slack_cost    = ΔSlack        = Slack(after) - Slack(before)
over_cost     = ΔOverWindow = Over(after)  - Over(before)
dist_cost     = ΔDistance     (近似差分)

slack_late     = 0.5
over_late(t)   = 10 * t^2
distance_late  = 0.5

cost =
  (-slack_late) * slack_cost
  + over_late(t) * over_cost
  + distance_late * dist_cost
  + rl_cost(remove_task, give_task)           # route_planner があれば
  + q_importance_weight * (Q(remove) - Q(give)) # route_planner があれば
```

符号の読み方:

- **slack_cost** は「余裕 (slack) が増えるほど +」になりやすい差分の取り方ですが、それに (**-slack_late**) を掛けて **slack増加を“コスト減”に変換**しています
- **over_cost** は遅れ（時間窓違反）が減るほど「負」になり、**over_late** が正なので **遅れ減少はコスト減**
- **dist_cost** は短くなるほど負になりやすい設計なので **距離短縮はコスト減**

3.3 スラック (Slack) 差分: `calculate_differ_slack(...)`

Slack の定義

`calculate_slacktime(pac_route)` は、`pac_task` ごとに

```
max(late_start_time - earliest_start_time, 0)
```

を足し合わせた総量です。

`earliest_start_time_list / latest_start_time_list` の概要

この Slack/Over の計算は、`MODEL3/VRPTW_functions.py` の以下で更新された時刻に依存します。

- `earliest_start_time_list(pac_list, dep_x, dep_y)`
 - 先頭から順に、（概ね）移動時間を足し込みながら `earliest_arrival_time` と `earliest_start_time` を更新します
 - 各タスクでは `earliest_start_time = max(到着時刻, ready_time)` の形を取ります
- `latest_start_time_list(pac_list)`
 - 末尾から逆順に、次タスクへ間に合うよう `late_start_time` を更新します

- 各タスクでは `late_start_time = min(next.late_start_time - 移動 - service_time, due_date)` の形を取ります

注意:

- これらはユークリッド距離 (`euclidean_distance`) を移動時間として扱う簡略モデルです (距離は `int(...)` に丸められます)
- `earliest_start_time_list` は先頭タスクで depot からの移動を厳密には反映していないため、厳密な実行可能性判定というより **差分比較のための近似指標** として使われていると解釈するのが安全です

差分の計算手順

`calculate_differ_slack(route, remove_task, add_task)` の実態は、`self.arrival_time_list` を基準にして以下を行います。

1. `before = Slack(self.arrival_time_list)`
2. `changed_list = deepcopy(self.arrival_time_list)`
3. `remove_task(remove_task, changed_list)` (`pac_task.task` が一致する要素を削除)
4. `add_task(add_task, changed_list)` (挿入位置推定して `pac_task(add_task)` を挿入)
5. `earliest_start_time_list(changed_list, dep_x, dep_y)`
6. `latest_start_time_list(changed_list)`
7. `after = Slack(changed_list)`
8. 返り値 `after - before`

注意:

- 差分評価が常に `self.arrival_time_list` を基準にしており、引数 `route` は本質的に使っていません
- 挿入位置推定 (`least_cost_time_insertion_index`) も `self.arrival_time_list` を参照するため、「仮に remove した後のルート」に対する厳密な挿入最適化ではなく、**近似評価** になっています

3.4 遅れ (Over Window) 差分: `caluculate_differ_over_window(...)`

Over の定義

`calculate_over_window(pac_route)` は、`pac_task` ごとに

```
max(earliest_start_time - due_date, 0)
```

を足し合わせた総量です (「開始時刻が期限を超えたぶん」)。

差分の計算手順

`caluculate_differ_over_window(...)` も Slack と同様に、`arrival_time_list` を基準に remove/add → earliest/latest 再計算 → `after - before` を返します。

3.5 距離差分 (近似): `calculate_differ_distance(taskA, taskB)`

距離差分は「局所エッジの増減」で近似しています (完全なルート再計算ではありません)。

- `taskA` (remove) がある場合:
 - `prev -> taskA` と `taskA -> next` を **引く**
- `taskB` (add) がある場合:
 - `least_cost_time_insertion_index(taskB)` で挿入位置 `index` を推定し、
 - `prev -> taskB` と `taskB -> next` を **足す**

この計算は、例えば「remove により `prev -> next` が新たに接続される」ぶんなどが明示的に入らないため、**厳密なΔ距離ではなくヒューリスティック**です。

3.6 進行度依存: `over_late = 10 * (n_steps/max_steps)^2`

`over_cost` (時間窓違反の差分) の重みが `t^2` で増えるため、

- 初期: 距離・slack など重視しつつ探索的
- 終盤: 遅れ (時間窓違反) 改善の優先度を上げる

という挙動を狙っています。

3.7 RL ルートプランナー補正: `_route_planner_cost_adjustment(...)`

`route_planner` が設定されている場合、交換前後のルートプランナーで評価し、その差分を `rl_cost` としてコストへ加算します。

- 重み (クラス定数):
 - `RL_REWARD_WEIGHT = 1.0`
 - `RL_UNASSIGNED_WEIGHT = 100000.0`
- 基準: `baseline_info = evaluate_route_with_planner(self.tasks)`
- 候補: `candidate_tasks = (remove_task を除いた tasks) + (add_task があれば追加)`
 - `_build_candidate_tasks_for_rl` がこの候補列を作ります
- 差分:
 - `reward_delta = candidate.total_reward - baseline.total_reward`
 - `unassigned_delta = candidate.unassigned_tasks - baseline.unassigned_tasks`
- 変換:
 - `rl_cost = - RL_REWARD_WEIGHT * reward_delta`
 - `rl_cost += RL_UNASSIGNED_WEIGHT * unassigned_delta`

意味:

- 報酬が増える (`reward_delta>0`) なら `rl_cost` は負 → **受諾しやすくなる**
- 未割当が増える (`unassigned_delta>0`) なら巨大な正の罰 → **強く拒否したい**

3.8 Q 重要度 (タスク重要度) 補正

さらに、プランナーが `discounted_q_task_scores(...)` を提供している場合、

```
cost += q_importance_weight * (Q(remove_task) - Q(give_task))
```

を加えます。

直感:

- $Q(\text{remove})$ が高い (重要タスクを手放す) → コスト増 (悪化) → 手放しにくい
- $Q(\text{give})$ が高い (重要タスクを受け取る) → コスト減 (改善) → 受け取りやすい

3.9 交渉中の評価 (VehicleNegotiator) との違い

このクラスの $\text{calculate_cost_saving}$ は主に **署名 (sign_contracts)** で使われます。一方で、交渉中 (NegMAS) の受諾判定は $\text{MODEL3/Vehicle_Negotiation.py}$ 側で行われ、そこで使われるコストは別実装です。

- 交渉中のコスト:
 - $\text{MODEL3/VRPTW_functions.py}$: $\text{calculate_cost_saving}(\text{arrival_time_list}, \text{taskA}, \text{taskB}, \text{tasks}, \text{bulletin_board})$
 - 係数や符号が一致しない (例: slack項の符号、 over_late の係数など)
 - RL の「ルート報酬差分」補正は入らず、主に Q 重要度 ($q_importance_weight$) の補正のみが加わる
- 署名時のコスト:
 - 本書で説明している
 $\text{Strategy_Vehicle_ver1.Vehicle.calculate_cost_saving(Agree)}$

結果として、**交渉でいったん成立した合意**が、署名時の評価軸ではそれほど良くない (または逆) というズレが起こります。

4. 挿入位置探索 $\text{least_cost_time_insertion_index}(\text{new_task: Task})$

この関数は「 new_task を self.tasks のどこに挿入するのが良いか」を推定し、挿入インデックスを返します。

4.1 目的 (距離増分の最小化)

候補位置ごとに「距離増分」を計算し、その最小を取ります。

- 先頭: $\text{dist}(\text{new_task}, \text{tasks}[0])$
- 末尾: $\text{dist}(\text{tasks}[-1], \text{new_task})$
- 中間:
 - $\text{dist}(\text{prev}, \text{new_task}) + \text{dist}(\text{new_task}, \text{next}) - \text{dist}(\text{prev}, \text{next})$

4.2 制約 (近似的な時間窓チェック)

$\text{is_within_time_window}(\text{new_task}, \text{prev_task}, \text{next_task})$ で候補位置をフィルタします。

- $\text{prev_task} / \text{next_task}$ は $\text{self.arrival_time_list}$ の要素 (pac_task)
- 判定は局所的で、概ね
 - 「prev の earliest_start から出発して new の due に間に合うか」
 - 「new の ready からサービスして next の due に間に合うか」

を見ています。

注意:

- depot から `new_task` への移動や「実際の到着時刻= $\max(\text{到着}, \text{ready})$ 」の扱いなどが完全には入らないため、これは厳密な可否判定というより **候補を粗く絞るための近似**です

5. ルート状態の初期化・更新

5.1 `first_step()` (ステップ冒頭の初期化)

`MODEL3/VRPTW-main.py` では毎ステップ `car.first_step()` が呼ばれます。

`Strategy_Vehicle_ver1.first_step` は次を行います。

1. `route_planner` があれば `plan_route(self.tasks, ...)` で並び替えを試みる
2. `arrival_time_list` を `pac_task(task)` の列として作り直す
3. `current_weight` を再集計
4. `earliest_start_time_list(arrival_time_list, dep_x, dep_y)`
5. `latest_start_time_list(arrival_time_list)`

このため、`calculate_cost_saving` が参照する `arrival_time_list` は、基本的にステップ冒頭で最新化されています。

5.2 `add_task / remove_task` (`pac_task` 列に対する操作)

コスト差分評価用に、`arrival_time_list` のコピーへ対して `remove/add` を行うための関数です。

- `remove_task(task, pac_route)`:
 - `pac.task == task` を満たす要素を削除
- `add_task(task, pac_route)`:
 - `least_cost_time_insertion_index` でインデックス推定して `pac_task(task)` を挿入
 - 推定できない場合は `due_date` 順のフォールバックで挿入位置を決める

5.3 `add(new_task) / remove(task)` (実ルート `self.tasks` の更新)

`add` は交渉実行フェーズで「相手から受け取ったタスク」をルートへ挿入するために使われます。

特徴:

- 容量制約を **進行度に応じて緩める** (序盤ほど少し過積載を許す) 形になっています:
 - `max_weight + 10 * (max_steps - n_steps) / max_steps`
- 挿入位置は `least_cost_time_insertion_index` を主に使い、無い場合は `due_date` によるフォールバックを使います
- `arrival_time_list` も同じインデックスで更新します (`pac_task(new_task)` を挿入)

`remove` は `self.tasks` と `arrival_time_list` の両方から該当要素を消し、`current_weight` を減らします。

5.4 `route_check(...)`

時間窓違反を検出するルート検出関数ですが、現状の挿入アルゴリズムでは **呼ばれていない (コメントアウトされた旧実装で使用予定) **形です。

5.5 is_task_assignable_with_or_tools(...)

`self` を取らない関数形でクラス内に定義されています（静的メソッド的）。

実施内容は「最大4タスクまで」「容量」「単純な挿入可否」を判定する簡易チェッカで、初期解生成や割当てで使う意図が読み取れます。

6. before_negotiation() と over_task (相手へ差し出す候補順)

`Vehicle_BASE.make_neg_agent()` は交渉開始時に `before_negotiation()` を呼び、`self.over_task` を `VehicleNegotiator.remove_list` として渡します。

`Strategy_Vehicle_ver1.before_negotiation` は次を行います。

1. 期限違反 (`earliest_start_time > due_date`) のタスクを `remove_list` に集める
2. 全タスクについて「そのタスクを remove したと仮定したコスト」を計算する
 - `cost[pac.task] = calculate_cost_saving(Agree(self, self, pac.task, None))`
3. `cost` を昇順にソートし、
 - `remove_list` に入っていたタスクを優先的に前へ
 - 残りをコスト順に後ろへ
4. その並びを `self.over_task` に保存する

意図:

- まず「明らかに時間窓違反を起こしているタスク」を手放したい
- それ以外も「手放すとコストが下がる（改善する）タスク」から順に候補にしたい

注意（実装上の挙動）:

- 手順3の中で `remove_list.remove(...)` を行うため、手順3の後半（「残りを後ろへ」）のループではすでに `remove_list` が空になりやすく、結果として **同じタスクが `over_task` に重複して入る可能性**があります（優先順位列としての意図は読み取れますが、現状のコードは「重複なしの列」を保証しません）。

7. まとめ (ver1 の狙いと挙動の特徴)

- 提案生成:
 - 少タスク: 全タスクを近傍へ提案（探索を広く）
 - 多タスク: K-means で過密領域を推定し、“外れ値タスク”を放出候補にして提案（局所最適化）
 - 容量逼迫時は重いタスクも放出候補へ追加
- 署名:
 - 合意を「自分が手放すタスク」単位に整理し、コストの良いものから拾う
 - 進行度で閾値を調整する設計だが、現状は **100000** がベースでかなり大きい
- コスト:
 - slack / 遅れ / 距離 の差分に加え、RL のルート評価差分と Q 重要度で補正できる
 - 遅れ（時間窓違反）の重みは終盤ほど強くなる (t^2)

8. 実装上の論点（分析メモ）

この節は「仕様というより、実装から読み取れる注意点」です。

- `check_offer()` が常に `True` を返すため、`VRPTW-main.py` 側の「交渉に入る前のフィルタ」が実質無効
- `find_available_vehicles()` の返り値型（車両オブジェクト）が、少タスク分岐のID前提処理と整合していない
- `calculate_differ_*` が `self.arrival_time_list` を基準にしており、仮ルートに対して厳密に最適化しているわけではない
- `insert_cost()` が `self.calculate_distance` を呼びますが、同クラス内に定義が見当たりません（未使用/未実装の可能性）
- `sign_contracts()` の容量チェック式は `swap` の一般形になっておらず、意図と違う可能性があります